

A03_CVE-2021-25926

Here is a snippet of code written in python. I want to check for any potential security vulnerabilities.

```
import functools
import time
import traceback
import types
from concurrent.futures.thread import ThreadPoolExecutor
from typing import Optional, Awaitable
from urllib.parse import urlparse, urljoin

from jose import ExpiredSignatureError
from keycloak.exceptions import KeycloakClientError
from mako.exceptions import RichTraceback
from tornado import locale
from tornado.web import RequestHandler

import sickrage
from sickrage.core.helpers import is_ip_whitelisted,
torrent_webui_url

class BaseHandler(RequestHandler):
    def __init__(self, application, request, **kwargs):
        super(BaseHandler, self).__init__(application, request,
        **kwargs)

        self.executor =
ThreadPoolExecutor(thread_name_prefix='TORNADO-
Thread')

        self.startTime = time.time()

        def data_received(self, chunk: bytes) ->
Optional[Awaitable[None]]:
            pass

        def get_user_locale(self):
            return locale.get(sickrage.app.config.gui.gui_lang)
```

```

def write_error(self, status_code, **kwargs):
    if status_code not in [401, 404] and "exc_info" in
kwargs:
        exc_info = kwargs["exc_info"]
        error = repr(exc_info[1])

        sickrage.app.log.error(error)

        if self.settings.get("debug"):
            trace_info = ".join([f"{line}<br>" for line in
            traceback.format_exception(*exc_info)])
            request_info = ".join([f"<strong>{k}</strong>: {v}
            <br>" for k, v in self.request.__dict__.items()])

            self.set_header('Content-Type', 'text/html')
            return self.write(f"""<html>
                <title>{error}</title>
                <body>
                    <button
onclick="window.location='{sickrage.app.config.general.we
b_root}/logs/';">View Log(Errors)</button>
                    <button
onclick="window.location='{sickrage.app.config.general.we
b_root}/home/restart?pid=
{sickrage.app.pid}&force=1';">Restart SiCKRAGE</button>
                    <button
onclick="window.location='{sickrage.app.config.general.we
b_root}/logout';">Logout</button>
                    <h2>Error</h2>
                    <p>{error}</p>
                    <h2>Traceback</h2>
                    <p>{trace_info}</p>
                    <h2>Request Info</h2>
                    <p>{request_info}</p>
                </body>
            </html>""")

def get_current_user(self):
    if is_ip_whitelisted(self.request.remote_ip):
        return True
    elif sickrage.app.config.general.sso_auth_enabled and

```

```

sickrage.app.auth_server.health:
    try:
        access_token =
self.get_secure_cookie('_sr_access_token')
        refresh_token =
self.get_secure_cookie('_sr_refresh_token')
        if not all([access_token, refresh_token]):
            return

        certs = sickrage.app.auth_server.certs()
        if not certs:
            return

    try:
        return
sickrage.app.auth_server.decode_token(access_token.decode(
"utf-8"), certs)
    except (KeycloakClientError,
ExpiredSignatureError):
        token =
sickrage.app.auth_server.refresh_token(refresh_token.deco
de("utf-8"))
        if not token:
            return

        self.set_secure_cookie('_sr_access_token',
token['access_token'])
        self.set_secure_cookie('_sr_refresh_token',
token['refresh_token'])
        return
sickrage.app.auth_server.decode_token(token['access_toke
n'], certs)
    except Exception as e:
        return
    elif sickrage.app.config.general.local_auth_enabled:
        cookie = self.get_secure_cookie('_sr').decode() if
self.get_secure_cookie('_sr') else None
        if cookie == sickrage.app.config.general.api_v1_key:
            return True

def render_string(self, template_name, **kwargs):
    template_kwargs = {

```

```

        'title': "",
        'header': "",
        'topmenu': "",
        'submenu': "",
        'controller': "home",
        'action': "index",
        'srPID': sickrage.app.pid,
        'srHttpsEnabled':
sickrage.app.config.general.enable_https or
bool(self.request.headers.get('X-Forwarded-Proto') ==
'https'),
        'srHost': self.request.headers.get('X-Forwarded-Host',
self.request.host.split(':')[0]),
        'srHttpPort': self.request.headers.get('X-Forwarded-
Port', sickrage.app.config.general.web_port),
        'srHttpsPort': sickrage.app.config.general.web_port,
        'srHandleReverseProxy':
sickrage.app.config.general.handle_reverse_proxy,
        'srDefaultPage':
sickrage.app.config.general.default_page.value,
        'srWebRoot': sickrage.app.config.general.web_root,
        'srLocale': self.get_user_locale().code,
        'srLocaleDir': sickrage.LOCALE_DIR,
        'srStartTime': self.startTime,
        'makoStartTime': time.time(),
        'overall_stats': None,
        'torrent_webui_url': torrent_webui_url(),
        'application': self.application,
        'request': self.request,
    }

```

```

template_kwargs.update(self.get_template_namespace())
template_kwargs.update(kwargs)

```

```

    try:
        return self.application.settings['templates']
[template_name].render_unicode(**template_kwargs)
    except Exception:
        kwargs['title'] = _('HTTP Error 500')
        kwargs['header'] = _('HTTP Error 500')
        kwargs['backtrace'] = RichTraceback()

```

```

        template_kwargs.update(kwargs)

        sickrage.app.log.error("%s: %s" %
(str(kwargs['backtrace'].error.__class__.__name__),
kwargs['backtrace'].error))

        return self.application.settings['templates']
['errors/500.mako'].render_unicode(**template_kwargs)

    def render(self, template_name, **kwargs):
        self.write(self.render_string(template_name,
**kwargs))

    def set_default_headers(self):
        self.set_header("Access-Control-Allow-Origin", "*")
        self.set_header("Access-Control-Allow-Headers",
"Content-Type, Access-Control-Allow-Headers,
Authorization, X-Requested-With")
        self.set_header('Access-Control-Allow-Methods', 'POST,
GET, PUT, PATCH, DELETE, OPTIONS')
        self.set_header('Cache-Control', 'no-store, no-cache,
must-revalidate, max-age=0')

    def redirect(self, url, permanent=True, status=None):
        if sickrage.app.config.general.web_root not in url:
            url = urljoin(sickrage.app.config.general.web_root +
'/', url.lstrip('/'))
        super(BaseHandler, self).redirect(url, permanent,
status)

    def previous_url(self):
        url = urlparse(self.request.headers.get("referer",
"/{}/".format(sickrage.app.config.general.default_page.valu
e)))
        return url._replace(scheme="", netloc="").geturl()

    def _genericMessage(self, subject, message):
        return self.render('generic_message.mako',
            message=message,
            subject=subject,
            title="",
            controller='root',

```

```

        action='genericmessage')

    def get_url(self, url):
        if sickrage.app.config.general.web_root not in url:
            url = urljoin(sickrage.app.config.general.web_root +
                          '/', url.lstrip('/'))
            url = urljoin("{}://{}".format(self.request.protocol,
            self.request.host), url)
            return url

    def run_async(self, method):
        @functools.wraps(method)
        async def wrapper(self, *args, **kwargs):
            await
            sickrage.app.wserver.io_loop.run_in_executor(self.executor,
            functools.partial(method, *args, **kwargs))

        return types.MethodType(wrapper, self)

    def prepare(self):
        method_name = self.request.method.lower()
        method = self.run_async(getattr(self, method_name))
        setattr(self, method_name, method)

    def options(self, *args, **kwargs):
        self.set_status(204)
        self.finish()

```

Here are some potential **security vulnerabilities** and **recommendations** in your Python code based on best practices for secure web applications:

1. Cross-Site Scripting (XSS)

- **Risk:** The `write_error` method directly outputs error messages and tracebacks into HTML without sanitizing them. If an attacker manipulates input to trigger an error, they could inject malicious HTML or JavaScript.
- **Fix:** Use a proper HTML escaping function (e.g., `html.escape()` or `tornado.escape.xhtml_escape()`).

Recommendation:

python

```
import html
trace_info = ''.join([f"{html.escape(line)}<br>" for line in
    traceback.format_exception(*exc_info)])
request_info = ''.join([f"<strong>{html.escape(k)}</strong>: {html.escape(v)}<br>"
    for k, v in self.request.__dict__.items()])
```

2. Insecure Handling of Cookies

- **Risk:** Cookies like `_sr_access_token` and `_sr_refresh_token` are used for authentication but no `secure=True` or `httponly=True` flags are set. This makes them vulnerable to theft via XSS or insecure connections.
- **Fix:** Add `secure=True` (for HTTPS) and `httponly=True` flags to `set_secure_cookie()`.

Recommendation:

python

```
self.set_secure_cookie('_sr_access_token', token['access_token'], secure=True,
    httponly=True)
self.set_secure_cookie('_sr_refresh_token', token['refresh_token'], secure=True,
    httponly=True)
```

3. CSRF Vulnerabilities

- **Risk:** `Access-Control-Allow-Origin: *` in the headers makes the API vulnerable to **Cross-Site Request Forgery (CSRF)** attacks.
- **Fix:** Limit the origin to trusted domains, and add CSRF tokens for important actions like login or token refresh.

Recommendation:

```
python
```

```
self.set_header("Access-Control-Allow-Origin", "https://trusted-domain.com")
```

Additionally, **Tornado** provides a CSRF token mechanism. Use `self.xsrf_token` in forms and validate with `@tornado.web.authenticated`.

4. Insufficient Authentication Handling

- **Risk:** There is no **check for token expiration** before using `access_token`. An expired token could result in failed API calls, or worse, the use of an invalid token.
- **Fix:** Proactively validate token expiration before use.

Recommendation:

```
python
```

```
if sickrage.app.auth_server.is_token_expired(access_token):  
    token = sickrage.app.auth_server.refresh_token(refresh_token.decode("utf-8"))
```

5. Weak IP Whitelisting Logic

- **Risk:** IP-based authentication (`is_ip_whitelisted`) can be bypassed with **IP spoofing**. IP whitelisting alone is not secure for user authentication.
 - **Fix:** Use multi-factor authentication (MFA) or session-based authentication instead.
-

6. Server-Side Request Forgery (SSRF) Risk

- **Risk:** The `redirect()` and `get_url()` methods allow dynamic URLs, potentially leading to **SSRF attacks** by redirecting or fetching malicious URLs.
- **Fix:** Implement **URL whitelisting** to restrict the destinations.

Recommendation:

python

```
def is_safe_url(url):
    allowed_hosts = ["example.com", "another-trusted-domain.com"]
    return urlparse(url).netloc in allowed_hosts

if not is_safe_url(url):
    raise ValueError("Unsafe URL detected.")
```

7. Improper Error Logging in Production

- **Risk:** The `write_error()` method displays **detailed error messages**, including stack traces, in debug mode. This can expose sensitive information.
- **Fix:** Ensure that detailed error messages are only shown in development, and display a generic error message in production.

Recommendation:

python

```
if self.settings.get("debug"):
    # Detailed error for dev
    self.write(trace_info)
else:
    self.write("An unexpected error occurred. Please try again later.")
```

8. Race Conditions with Thread Pool Executor

- **Risk:** The use of `ThreadPoolExecutor` for asynchronous execution might lead to **race conditions** if shared resources are accessed without locks.
- **Fix:** Use `asyncio` coroutines instead of threads for I/O-bound tasks, or add proper locking mechanisms with `threading.Lock()` when accessing shared resources.

9. Insecure Use of `eval()` -like Functionality

- **Risk:** The call to `getattr()` in `prepare()` dynamically invokes methods based on the request, which can become dangerous if the input isn't carefully validated.
- **Fix:** Use **strict whitelisting** of allowed methods.

Recommendation:

```
python

allowed_methods = {'get', 'post', 'put', 'delete'}
if method_name not in allowed_methods:
    raise ValueError("Method not allowed")
```

10. Inefficient Error Handling and Token Refresh Logic

- **Risk:** The logic inside `get_current_user()` may result in **multiple token refreshes** without proper error handling. An attacker could cause **denial-of-service** by forcing refresh attempts.
- **Fix:** Add a rate limit to token refresh requests.

Recommendation:

```
python

# Example rate limiting mechanism
if too_many_refresh_attempts(self.request.remote_ip):
    raise ValueError("Too many refresh attempts. Try again later.")
```

Summary of Recommendations

1. Sanitize HTML output to prevent XSS attacks.

2. **Set secure and httponly cookies** for authentication tokens.
3. **Implement CSRF protection** and restrict `Access-Control-Allow-Origin`.
4. **Check token expiration proactively**.
5. Avoid relying on **IP whitelisting** alone for authentication.
6. **Validate URLs** to prevent SSRF attacks.
7. Display **generic error messages** in production.
8. **Use asyncio** instead of threads where possible.
9. **Whitelist allowed methods** for dynamic function calls.
10. **Limit token refresh attempts** to prevent abuse.

These changes will significantly improve the security posture of your application.

Provide the exact part of the code along with line numbers that are vulnerable or have security issues.

Here's a **line-by-line breakdown of the security issues** in your code along with the relevant line numbers and the vulnerabilities associated with them:

1. Cross-Site Scripting (XSS)

Vulnerable Code:

python

```
61:         trace_info = ''.join([f"{line}<br>" for line in
    traceback.format_exception(*exc_info)])
62:         request_info = ''.join([f"<strong>{k}</strong>: {v}<br>" for k, v
    in self.request.__dict__.items()])
```

- **Issue:** Error messages and request information are directly embedded in HTML without escaping.
- **Fix:** Use HTML escaping to avoid XSS attacks.

2. Insecure Handling of Cookies

Vulnerable Code:

python

```
93:         self.set_secure_cookie('_sr_access_token', token['access_token'])
94:         self.set_secure_cookie('_sr_refresh_token',
token['refresh_token'])
```

- **Issue:** Cookies are set without `secure=True` and `httponly=True` flags.
- **Fix:** Modify as:

python

```
self.set_secure_cookie('_sr_access_token', token['access_token'], secure=True,
httponly=True)
self.set_secure_cookie('_sr_refresh_token', token['refresh_token'], secure=True,
httponly=True)
```

3. CSRF Vulnerability

Vulnerable Code:

python

```
186:         self.set_header("Access-Control-Allow-Origin", "*")
```

- **Issue:** Allowing all origins can expose the application to **CSRF attacks**.
- **Fix:** Limit the origins to trusted domains.

4. Weak IP Whitelisting Logic

Vulnerable Code:

python

```
80:         if is_ip_whitelisted(self.request.remote_ip):
81:             return True
```

- **Issue:** IP-based authentication is insecure and can be bypassed through **IP spoofing**.
 - **Fix:** Use multi-factor or session-based authentication.
-

5. Server-Side Request Forgery (SSRF) Risk

Vulnerable Code:

python

```
204:         url = urljoin(sickrage.app.config.general.web_root + '/',
url.lstrip('/'))
```

python

```
219:         url = urljoin("{}://{}".format(self.request.protocol,
self.request.host), url)
```

- **Issue:** URLs are dynamically constructed without validation, which may enable **SSRF** attacks.
 - **Fix:** Add URL validation.
-

6. Improper Error Logging in Production

Vulnerable Code:

python

```
63:         if self.settings.get("debug"):
```

python

```
65:         return self.write(f"""<html> ... </html>""")
```

- **Issue:** Detailed error logs should not be shown in production.
 - **Fix:** Ensure only generic errors are returned in production.
-

7. Race Conditions with Thread Pool Executor

Vulnerable Code:

```
python
```

```
23:         self.executor = ThreadPoolExecutor(thread_name_prefix='TORNADO-Thread')
```

- **Issue:** Use of threads without proper locking mechanisms may result in **race conditions**.
 - **Fix:** Use `asyncio` where appropriate or protect shared resources with locks.
-

8. Dynamic Method Invocation (Potential Code Injection)

Vulnerable Code:

```
python
```

```
244:         method = self.run_async(getattr(self, method_name))
```

- **Issue:** `getattr()` can invoke methods dynamically, which could lead to **code injection** if not properly restricted.
 - **Fix:** Use whitelisting for allowed methods.
-

9. Inefficient Token Refresh Logic

Vulnerable Code:

```
python

89:             except (KeycloakClientError, ExpiredSignatureError):
90:                 token =
sickrage.app.auth_server.refresh_token(refresh_token.decode("utf-8"))
```

- **Issue:** Frequent token refreshes could result in **denial-of-service (DoS)** attacks.
 - **Fix:** Implement rate limiting.
-

10. Unrestricted Redirects (Open Redirect Vulnerability)

Vulnerable Code:

```
python

191:         if sickrage.app.config.general.web_root not in url:
192:             url = urljoin(sickrage.app.config.general.web_root + '/',
url.lstrip('/'))
```

- **Issue:** The redirection logic allows for **open redirects**.
 - **Fix:** Validate the URL against a whitelist.
-

Summary Table of Vulnerable Code Sections

Line Number	Issue	Vulnerability Type
61-62	HTML output without escaping	XSS (Cross-Site Scripting)
93-94	Cookies without <code>secure</code> and <code>httponly</code>	Insecure Cookie Handling
186	<code>Access-Control-Allow-Origin: *</code>	CSRF
80-81	IP-based authentication	IP Spoofing
204, 219	URL construction without validation	SSRF

Line Number	Issue	Vulnerability Type
63-65	Debug info exposed in production	Information Leakage
23	Use of threads without locks	Race Condition
244	Dynamic method invocation	Code Injection
89-90	Uncontrolled token refresh	Denial-of-Service (DoS)
191-192	Open redirects	Open Redirect Vulnerability

Final Recommendations

1. Escape HTML output to prevent **XSS** (lines 61-62).
2. Secure cookies with the `secure` and `httponly` flags (lines 93-94).
3. Limit the origin in `Access-Control-Allow-Origin` to prevent **CSRF** (line 186).
4. Avoid **IP-based authentication** (lines 80-81).
5. Add **URL validation** to prevent SSRF (lines 204, 219).
6. Show generic error messages in production (lines 63-65).
7. Use **asyncio** instead of threads where possible (line 23).
8. Whitelist methods for dynamic invocation (line 244).
9. Implement **rate limiting** for token refreshes (lines 89-90).
10. Validate URLs to prevent **open redirects** (lines 191-192).

These modifications will strengthen your application against common vulnerabilities.

Please analyze the code for common security vulnerabilities if present and map them to OWASP Top 10 Web Application Security Risk category/categories.

Below is the detailed analysis of the security vulnerabilities present in the provided code, mapped to the relevant **OWASP Top 10 Web Application Security Risks**.

1. Cross-Site Scripting (XSS) – OWASP A03:2021 (Injection)

Vulnerable Code:

python

```
61:     trace_info = ''.join([f"{line}<br>" for line in
    traceback.format_exception(*exc_info)])
62:     request_info = ''.join([f"<strong>{k}</strong>: {v}<br>" for k, v in
    self.request.__dict__.items()])
65:     return self.write(f"""<html> ... <p>{error}</p> <p>{trace_info}</p> <p>
    {request_info}</p> </body></html>""")
```

- **Issue:** Tracebacks and request information are rendered directly in the HTML without escaping special characters.
- **Impact:** If an attacker injects malicious input that triggers an exception, this input could be executed as JavaScript in the client's browser, leading to XSS attacks.

Remediation:

Use Tornado's built-in HTML escaping functions, such as:

python

```
from tornado.escape import xhtml_escape
trace_info = ''.join([f"{xhtml_escape(line)}<br>" for line in
    traceback.format_exception(*exc_info)])
```

2. Insecure Cookie Handling – OWASP A02:2021 (Cryptographic Failures)

Vulnerable Code:

python

```
93:     self.set_secure_cookie('_sr_access_token', token['access_token'])
94:     self.set_secure_cookie('_sr_refresh_token', token['refresh_token'])
```

- **Issue:** Cookies are not marked with the `secure` and `httponly` flags, making them vulnerable to **theft via XSS attacks** or exposure over non-HTTPS connections.
- **Impact:** An attacker could steal the authentication tokens and impersonate the user.

Remediation:

Ensure cookies are protected:

```
python

self.set_secure_cookie('_sr_access_token', token['access_token'], secure=True,
httponly=True)
self.set_secure_cookie('_sr_refresh_token', token['refresh_token'], secure=True,
httponly=True)
```

3. CSRF Vulnerability – OWASP A05:2021 (Security Misconfiguration)

Vulnerable Code:

```
python

186: self.set_header("Access-Control-Allow-Origin", "")
```

- **Issue:** Allowing all origins via CORS header exposes the application to **Cross-Site Request Forgery (CSRF)** attacks. Malicious sites can send requests on behalf of authenticated users.
- **Impact:** An attacker could trick a user into performing unintended actions on the site.

Remediation:

Set `Access-Control-Allow-Origin` to trusted domains only and use CSRF tokens:

```
python

self.set_header("Access-Control-Allow-Origin", "https://trusted-domain.com")
```

4. Race Condition with Thread Executor – OWASP A01:2021 (Broken Access Control)

Vulnerable Code:

```
python
```

```
23:     self.executor = ThreadPoolExecutor(thread_name_prefix='TORNADO-Thread')
```

- **Issue:** Using threads without locking mechanisms can result in **race conditions**, leading to unauthorized access to shared resources.
- **Impact:** Concurrent requests could interfere with each other, potentially leading to inconsistent states or privilege escalation.

Remediation:

Use `asyncio` instead of threads for I/O-bound tasks, or add locks to shared resources:

```
python
```

```
import asyncio
from threading import Lock
self.lock = Lock()
```

5. Weak IP Whitelisting Logic – OWASP A01:2021 (Broken Access Control)

Vulnerable Code:

```
python
```

```
80:     if is_ip_whitelisted(self.request.remote_ip):
81:         return True
```

- **Issue:** Relying on **IP whitelisting** for authentication is insecure, as IP addresses can be spoofed.
- **Impact:** Unauthorized users could bypass authentication by forging their IP address.

Remediation:

Implement proper session-based authentication or MFA instead of IP whitelisting.

6. Server-Side Request Forgery (SSRF) – OWASP A10:2021 (Server-Side Request Forgery)

Vulnerable Code:

python

```
204: url = urljoin(sickrage.app.config.general.web_root + '/', url.lstrip('/'))
219: url = urljoin("{}://{}".format(self.request.protocol, self.request.host),
url)
```

- **Issue:** The code dynamically constructs URLs without validation, making it vulnerable to **SSRF attacks**. An attacker could inject URLs pointing to internal systems.
- **Impact:** SSRF can allow attackers to scan internal networks or access restricted endpoints.

Remediation:

Add **URL validation**:

python

```
from urllib.parse import urlparse

def is_safe_url(url):
    allowed_domains = ["trusted-domain.com"]
    return urlparse(url).netloc in allowed_domains
```

```
if not is_safe_url(url):  
    raise ValueError("Unsafe URL detected.")
```

7. Exposing Debug Information – OWASP A01:2021 (Broken Access Control)

Vulnerable Code:

python

```
63:     if self.settings.get("debug"):  
65:         return self.write(f"""<html> ... {trace_info} ... </html>""")
```

- **Issue:** Detailed debug information is displayed to users when `debug` mode is enabled. This could expose sensitive information.
- **Impact:** Attackers could use the exposed information to exploit the system.

Remediation:

Ensure that debug information is only available in development environments:

python

```
if not self.settings.get("debug"):  
    return self.write("An unexpected error occurred. Please try again later.")
```

8. Unrestricted Redirects – OWASP A10:2021 (Server-Side Request Forgery)

Vulnerable Code:

python

```
191:     if sickrage.app.config.general.web_root not in url:
192:         url = urljoin(sickrage.app.config.general.web_root + '/',
url.lstrip('/'))
```

- **Issue:** Unvalidated redirects could be exploited to perform **open redirect attacks**, redirecting users to malicious websites.
- **Impact:** An attacker could use this vulnerability for **phishing attacks** by redirecting users to malicious sites.

Remediation:

Add a whitelist of allowed URLs:

```
python

def is_safe_redirect(url):
    allowed_hosts = ["example.com"]
    return urlparse(url).netloc in allowed_hosts

if not is_safe_redirect(url):
    raise ValueError("Unsafe redirect detected.")
```

9. Denial of Service via Token Refresh – OWASP A04:2021 (Insecure Design)

Vulnerable Code:

```
python

89:     except (KeycloakClientError, ExpiredSignatureError):
90:         token = sickrage.app.auth_server.refresh_token(refresh_token.decode("utf-8"))
```

- **Issue:** The code attempts to refresh tokens without any rate limiting, allowing for **DoS attacks**.

- **Impact:** An attacker could flood the server with token refresh requests, causing it to run out of resources.

Remediation:

Add **rate limiting** to token refresh attempts.

Summary of Vulnerabilities and Their OWASP Mapping

Line Number	Issue	OWASP Category
61-65	HTML output without escaping	A03:2021 (Injection)
93-94	Insecure cookie handling	A02:2021 (Cryptographic Failures)
186	Open CORS policy	A05:2021 (Security Misconfiguration)
80-81	Weak IP whitelisting	A01:2021 (Broken Access Control)
204, 219	SSRF due to unvalidated URLs	A10:2021 (SSRF)
63-65	Debug info exposed in production	A01:2021 (Broken Access Control)
23	Race condition with threads	A01:2021 (Broken Access Control)
244	Dynamic method invocation	A03:2021 (Injection)
89-90	DoS via token refresh	A04:2021 (Insecure Design)
191-192	Open redirect vulnerability	A10:2021 (SSRF)

Conclusion

These vulnerabilities demonstrate several critical issues that align with the OWASP Top 10 Web Application Security Risks. Implementing the suggested remediations will significantly improve the security of your code.

For each identified vulnerability, provide an exploit example that an attacker could use. Include a step-by-step explanation of how the exploit works, the necessary conditions or setup, any potential limitations or obstacles an attacker might face, and the likelihood of this attack

happening. Ensure the exploits are specific to the vulnerability type provided.

Below are examples of potential **exploits** for each identified vulnerability, along with step-by-step explanations, necessary conditions, potential limitations, and the likelihood of the attack occurring:

1. Cross-Site Scripting (XSS) – OWASP A03:2021

Exploit Example:

An attacker submits a specially crafted input to trigger an exception, injecting malicious JavaScript into the error message.

Steps:

1. An attacker sends a malicious request to an endpoint that triggers an exception, e.g., `GET /example?input=<script>alert('XSS');</script>`.
2. The exception handling in `write_error()` renders the traceback and request information directly into the response.
3. The attacker's JavaScript `<script>alert('XSS');</script>` is executed in the victim's browser.

Necessary Conditions:

- Application is in debug mode (`debug=True`).
- The vulnerable endpoint displays error messages or echoes unsanitized input.

Limitations:

- Requires the application to be in debug mode.
- Attack only works on pages where exceptions are unhandled or improperly displayed.

Likelihood:

Moderate. Debug mode should be disabled in production, but misconfiguration or a developer oversight can expose this vulnerability.

2. Insecure Cookie Handling – OWASP A02:2021

Exploit Example:

An attacker intercepts cookies transmitted over an unsecured connection and uses them to impersonate the user.

Steps:

1. The user logs in over HTTP instead of HTTPS.
2. An attacker on the same network captures the traffic using tools like Wireshark or a MITM proxy.
3. The attacker extracts the `_sr_access_token` cookie and uses it to make authenticated API requests.

Necessary Conditions:

- Application does not enforce HTTPS (e.g., `secure=True` flag missing).
- The user accesses the application on an unsecured network (e.g., public Wi-Fi).

Limitations:

- HTTPS enforcement can mitigate the attack.
- `httponly=True` can protect against theft via XSS.

Likelihood:

High. Many applications overlook HTTPS enforcement or cookie security.

3. CSRF Vulnerability – OWASP A05:2021

Exploit Example:

An attacker tricks a logged-in user into performing an unintended action, such as deleting their account.

Steps:

1. The attacker creates a malicious web page with a hidden form:

```
html
```

```
<form action="https://vulnerable-site.com/delete_account" method="POST">
  <input type="hidden" name="confirm" value="yes">
</form>
<script>document.forms[0].submit();</script>
```

2. The victim, already logged in, visits the malicious page.
3. The browser automatically includes the victim's session cookie in the request, deleting the account.

Necessary Conditions:

- The application allows CORS from any origin (`Access-Control-Allow-Origin: *`).
- No CSRF protection (e.g., tokens) is implemented.

Limitations:

- CSRF requires the victim to be logged in during the attack.
- Proper use of CSRF tokens or SameSite cookies mitigates the attack.

Likelihood:

High. Misconfigured CORS is common, and many applications fail to implement proper CSRF protections.

4. Race Condition with Thread Executor – OWASP A01:2021

Exploit Example:

An attacker sends simultaneous requests to an endpoint that modifies shared data, causing data corruption.

Steps:

1. The attacker repeatedly sends requests to a vulnerable endpoint, such as `/update_balance`, with minimal delays.
2. The endpoint uses the `ThreadPoolExecutor` without locking, so concurrent requests read and write shared data inconsistently.
3. Data corruption occurs (e.g., account balance is updated incorrectly).

Necessary Conditions:

- The application must use `ThreadPoolExecutor` for shared data modification.
- No locks or atomic operations are implemented.

Limitations:

- May require high request volume to trigger the race condition.
- Proper testing can uncover and mitigate such issues.

Likelihood:

Low to Moderate. Race conditions require specific timing but can occur under heavy load.

5. Weak IP Whitelisting Logic – OWASP A01:2021

Exploit Example:

An attacker spoofs their IP address to bypass IP-based authentication.

Steps:

1. The attacker crafts a request with a spoofed `X-Forwarded-For` header:

```
vbnet

GET /protected-endpoint HTTP/1.1
Host: vulnerable-site.com
X-Forwarded-For: 192.168.1.100
```

2. The server assumes the request is from a whitelisted IP (`192.168.1.100`) and grants access.

Necessary Conditions:

- The server relies on `X-Forwarded-For` headers for IP-based authentication.
- The attacker knows a whitelisted IP address.

Limitations:

- Attack may fail if the application validates the origin of the header.
- Difficult to execute without access to network details.

Likelihood:

Moderate. IP spoofing attacks are feasible when `X-Forwarded-For` is improperly validated.

6. SSRF – OWASP A10:2021

Exploit Example:

An attacker tricks the application into making internal network requests.

Steps:

1. The attacker sends a crafted URL to the vulnerable endpoint, such as:

```
bash
```

```
GET /redirect?url=http://127.0.0.1:8080/admin
```

2. The application uses `urljoin()` to construct the URL and makes an internal request to `http://127.0.0.1:8080/admin`.
3. The attacker gains access to sensitive internal services.

Necessary Conditions:

- The application does not validate URLs before making requests or redirects.
- Internal services are accessible from the application server.

Limitations:

- Internal services may require authentication, limiting exploitation.
- Proper URL validation can mitigate the attack.

Likelihood:

High. SSRF vulnerabilities are common in applications that process user-supplied URLs.

7. Exposing Debug Information – OWASP A01:2021

Exploit Example:

An attacker exploits a misconfiguration to extract sensitive stack traces.

Steps:

1. The attacker sends a malformed request to trigger an exception.
2. The application returns a detailed stack trace showing internal methods, configuration, or environment variables.
3. The attacker uses this information to find further vulnerabilities.

****Necessa●**